Introduction

I created an image processing application that explores real-time image processing techniques such as colour space transformations, colour channels thresholding, and face detection. Filters like blurring and pixelation are implemented to anonymize faces. Lastly, two unique extensions of my own.

Implementation and Findings

The application captures a live webcam feed and resizes the frames to 160x120 pixels. I included a snapshot feature, allowing users to capture a still frame for analysis. The first transformation applied is grayscale conversion, followed by a 20% brightness increase, ensuring no pixel exceeds an intensity of 255.

Next, I separated the image into red, green, and blue channels and applied thresholding to each. Users can adjust the threshold values using sliders to see how different channels influence segmentation.

Findings from Thresholding

-Red channel: Highlights warm tones and bright areas, useful for detecting skin tones. However, it produced high contrast, sometimes making objects appear too sharp. Noise was noticeable in areas with less red, making it less stable in mixed-colour images.

- Green Channel: Since green contributes the most to overall brightness, thresholding in this channel retained more image details while minimizing noise. This is the most reliable for segmenting objects clearly without excessive loss of information.

- Blue Channel: Most fragmented and noisy thresholding. Blue, the least dominant colour in natural lighting, struggled to provide clear segmentation, leading to a noisier output. However, it was effective for detecting blue-tinted objects in the image.

- YCbCr : Improved object separation, especially in differentiating foreground from background. The chromatic components allowed for a more refined segmentation process, making it effective for complex color variations.

- HSV : Useful in reducing noise. By focusing on hue rather than intensity, HSV thresholding helped separate objects based on colour distinctions rather than brightness levels, resulting in smoother segmentation.

Face Detection and Privacy Filters

I integrated real-time face detection using ml5.js, which places bounding boxes around detected faces. I implemented the filters:

- Grayscale conversion

- Blurring to obscure details

- YCbCr colour conversion for an alternative representation

- Pixelation by averaging pixel blocks to distort facial features

Challenges and Solutions

1. Maintaining smooth real-time performance while handling multiple transformations. I optimized efficiency by reusing processed images instead of recalculating transformations.

2. Facial detection wasn't working properly. Initially, the face detection model struggled to recognize my face even under varying lighting conditions, angles . To improve accuracy, I adjusted the lighting in my environment and ensured the webcam had a clear view.

Extension

1. Emoji-Based Facial detection:

This feature makes the project more interactive by detecting facial expressions (e.g: happy, sad) and replacing faces with corresponding emojis. Unlike traditional privacy filters that simply obscure faces, this feature makes the application more dynamic and engaging.

2. Kaleidoscope Effect:

The kaleidoscope effect brings an artistic element to the application. Instead of using static filters, this feature continuously shifts and mirrors image segments while adjusting color hues. The effect changes as users move, creating symmetrical patterns.

This sets my project apart by blending technical image processing with creative expression.